

Project LEE

Architecture Refactor Command for Replit

API-first Lamont Labs systems model | CIL and CerbaSeal externalization | Replit AI continuity

Use this document as the implementation instruction. Preserve current working functionality and refactor authority boundaries rather than rebuilding Project LEE.

Primary Instruction

Refactor the current Project LEE architecture to enforce the new API-first Lamont Labs systems model.

Do not rebuild Project LEE from scratch. Preserve the current working Console, Android companion, Windows desktop packaging, PostgreSQL data model, Event Log, Executive Loop, System Contract, Capability Registry, backups, release pipeline, and existing working functionality.

The goal is to cleanly separate LEE from specialist Lamont Labs systems and make future systems connect through standardized APIs.

1. CIL Must Be Completely External

CIL is an independent Lamont Labs system.

Project LEE may contain only what is required to communicate with CIL:

- API client
- System Contract
- request/response schemas
- authentication/signing
- health checks
- endpoint configuration
- timeout/retry behavior
- correlation IDs
- service status
- normalized events
- UI/service-health representation

LEE must not contain duplicate CIL decision logic.

Remove or refactor any local LEE implementation that duplicates CIL responsibilities, including where applicable:

- T1 reuse logic
- T2 semantic reuse logic
- T3 escalation decision logic
- cognitive asset selection
- contradiction arbitration
- drift handling
- model-selection intelligence
- CIL-specific cost optimization
- CIL crystallization/governance logic

CIL is authoritative for cognitive routing.

2. CIL Must Be Consulted Before Model Selection

For every normal LEE-originated AI/reasoning request, enforce this architecture:

Identity
 -> **Constitution**
 -> **Intent**
 -> **Query Engine**
 -> **Context Economy**
 -> **CIL API**

CIL then decides:

- T1 governed reuse
- T2 governed reuse
- new reasoning required
- model/model class to use
- approved reasoning route
- HOLD / no safe route where applicable

If CIL resolves through T1/T2, no external model call occurs.

If CIL determines a new model call is required, only then continue to the Model Router.

There must be no normal path where LEE chooses a paid/external model before CIL evaluates the request.

Add architecture and behavioral tests proving this ordering.

3. Narrow the Model Router

The LEE Model Router remains, but it is execution infrastructure, not the cognitive/model-selection authority.

CIL chooses the route. The Model Router executes CIL's decision.

The Model Router may handle:

- endpoint selection
- transport
- request formatting
- response normalization
- usage capture
- permitted retries
- execution telemetry

It must not silently choose a different model.

If the CIL-selected route fails, return that failure to CIL for a new routing decision.

4. CerbaSeal Must Be Completely External

CerbaSeal is an independent Lamont Labs governance system.

LEE may contain only:

- API client
- System Contract
- request/response schemas
- authentication/signing
- health checks
- endpoint configuration
- timeout behavior
- correlation IDs
- service status
- decision references
- normalized events
- UI/governance representation

LEE must not contain duplicate CerbaSeal decision logic.

Remove or refactor any LEE implementation that duplicates CerbaSeal responsibilities, including where applicable:

- policy-pack evaluation
- ALLOW/HOLD/REJECT decision logic
- approval-chain evaluation
- CerbaSeal invariants
- decision-envelope creation
- evidence-bundle generation
- authorization-expiry logic owned by CerbaSeal

CerbaSeal is authoritative for consequential-action governance.

If a capability requires CerbaSeal and CerbaSeal is unavailable: HOLD.

There must be no local LEE governance fallback that can authorize the action.

5. Generalize the Universal Systems API Layer

Use the existing System Contract and Capability Registry as the foundation rather than creating redundant architecture.

Generalize them into a Universal Systems API Layer that can support CIL, CerbaSeal, Replit AI Bridge, Greyline, QuantraCore, and future Lamont Labs systems.

A connected system should be describable through a versioned machine-readable contract containing:

- system identity
- version

- description
- runtime location
- API endpoint
- authentication method
- health contract
- capabilities
- request schemas
- response schemas
- emitted events
- permissions
- risk classification
- CerbaSeal requirement
- human-confirmation requirement
- dependencies
- economics/cost metadata
- availability state

Do not hardcode future projects into LEE Core.

LEE should understand registered capabilities rather than project-specific internal implementations.

6. Support Capability-Level Connection Modes

The Universal Systems API Layer should support:

- OBSERVE
- USE
- MANAGE
- GOVERNED_MANAGE

Authority applies per capability.

A system may expose some read-only capabilities and other managed/governed capabilities.

7. Preserve the Replit AI Bridge

The desktop migration must preserve Replit-managed AI access.

The long-term AI execution path must support:

LEE Core on K6
 -> **CIL API**
 -> **CIL selects model/model class**
 -> **Model Router**
 -> **Replit AI Bridge**
 -> **Replit-managed AI integration**

-> **selected model**
-> **result back to LEE**

The Replit AI Bridge exists so desktop LEE does not need separate OpenAI, Anthropic, Gemini, or other provider API keys whenever Replit can supply the required managed model integration.

The Replit AI Bridge must not independently choose or silently substitute models.

If its requested model route is unavailable, return the failure to CIL.

8. Preserve Desktop Architecture

Do not regress the existing Windows desktop work.

Preserve:

- Electron desktop shell
- K6/local-first runtime design
- private local PostgreSQL
- migrations
- local Brain
- Event Log
- tray process
- supervisor
- desktop icon/shortcuts
- NSIS installer
- GitHub Release pipeline
- restart persistence
- software-version vs Brain-version separation

CIL, CerbaSeal, Replit AI Bridge, and other specialist systems remain external API-connected services and must not be bundled as duplicate implementations inside the desktop package.

9. Preserve the Android Companion

Android remains a first-class companion to the same authoritative LEE Brain.

Normal architecture:

Android
-> **LEE Core**

-> **Universal Systems API Layer / CIL / CerbaSeal / Replit AI Bridge as required**

The Android app should not independently hold credentials or direct integrations for CIL, CerbaSeal, Greyline, QuantraCore, or AI providers.

Preserve:

- Today
- Ask Lee

- Capture
- local/offline capture
- Brief
- Waiting
- Alerts
- Approvals
- system status
- lightweight governed controls

There must still be one authoritative LEE Brain.

10. Update Architectural Terminology

Replace the concept of:

Layer 7 - Internal Capability Services

with terminology reflecting that these are independent systems, such as:

Connected Lamont Labs Systems / Universal Systems API Layer

CIL and CerbaSeal must no longer be described as internal LEE engines or internal implementations.

11. Preserve Responsibility Boundaries

LEE

- memory
- context
- projects
- people
- objectives
- operational intelligence
- Executive Loop
- portfolio intelligence
- orchestration
- system supervision
- history
- learning

CIL

- cognitive reuse
- reasoning resolution
- AI cost minimization
- model/model-class selection
- cognitive routing

Model Router

- executes CIL-selected model routes

Replit AI Bridge

- executes approved Replit-managed AI calls

CerbaSeal

- consequential-action governance
- ALLOW/HOLD/REJECT

Greyline / QuantraCore / future systems

- own their specialist domain logic

12. Required Tests

1. LEE cannot invoke a normal paid/external model before CIL.
2. T1/T2 CIL resolution prevents unnecessary model execution.
3. T3 CIL result selects the model route.
4. Model Router executes but does not override CIL's decision.
5. Replit AI Bridge cannot silently substitute models.
6. Failed model execution returns to CIL for rerouting.
7. LEE has no direct CIL database access.
8. LEE has no direct CerbaSeal database access.
9. No duplicate CIL decision logic remains active inside LEE.
10. No duplicate CerbaSeal authorization logic remains active inside LEE.
11. CerbaSeal-required actions fail closed when CerbaSeal is unavailable.
12. A registered test system can be added through the Universal Systems API Layer without changing LEE Core.
13. Android routes external-system operations through LEE Core.
14. Desktop packaging still builds and passes its existing Windows validation.

13. Documentation Updates

Update:

- Project LEE Manual
- architecture map
- System Contract documentation
- glossary
- System Manifest terminology

- current project description
- relevant developer documentation

Document these permanent invariants:

CIL and CerbaSeal are independent Lamont Labs systems.

LEE communicates with them strictly through authenticated APIs.

LEE contains integration contracts, not duplicate specialist implementations.

CIL is authoritative for AI reasoning reuse and model selection.

Normal LEE AI requests go through CIL before Replit or another external model route.

CerbaSeal is authoritative for consequential-action governance.

Future Lamont Labs systems connect through standardized APIs rather than being coded into LEE Core.

Final Requirement

Make this an architectural refactor, not a feature expansion.

Preserve current working behavior wherever it does not conflict with these new authority boundaries.

After implementation, run the complete behavioral test suite, desktop packaging validation, System Manifest checks, and architecture-boundary tests.

Then give me a final report showing:

- files changed
- duplicate CIL logic removed
- duplicate CerbaSeal logic removed
- new Universal Systems API architecture
- final AI request path
- final consequential-action path
- desktop validation result
- Android validation result
- test results
- remaining limitations
- migration concerns

Do not claim completion unless the new authority boundaries are enforced in runtime code and behavioral tests, not merely documented.